FloodWatch: lightweight U-Nets for flood-water segmentation from Sentinel-1 radar

A controlled study of input channels, loss and encoder on Sen1Floods11, trained on a 4 GB laptop GPU

Roopak Raam · Mini-project report, November 2026

1 Introduction

Floods are the most frequent kind of disaster. Of the 7,348 major disaster events recorded worldwide between 2000 and 2019, 3,254 were floods [1], and satellite records show that the share of the world's population living in places that flood has grown [2]. When a flood is under way, the first thing responders need is a map of where the water is. Satellites can provide one for a whole region within hours, if the water can be seen.

Seeing it is the difficulty. Floods arrive with heavy cloud, and an optical satellite cannot see through cloud. Synthetic aperture radar (SAR) can. A radar satellite sends its own microwave pulses and records what is scattered back, so it works at night and through cloud. The Sentinel-1 mission images most land every 6 to 12 days at 10 m resolution and its data are free [3]. Calm water is smooth: it reflects the side-looking radar pulse away from the satellite, so open water appears dark in a radar image. This is the physical basis of every radar flood map.

"Dark means water" is, however, only a rule of thumb. Tarmac, sand, radar shadow and some crops are dark too, while water roughened by wind or broken by vegetation can be as bright as land. Operational systems started from thresholds on the backscatter [4, 5]; over the last years convolutional networks, above all U-Nets [6], have taken over because they can use the spatial context around a pixel. Sen1Floods11 [7] made this research comparable by publishing Sentinel-1 chips of eleven flood events with water masks, 446 of them labelled by hand.

Much of the published work on this dataset uses large models: ResNet-50 encoders [8], or foundation models with hundreds of millions of parameters [9, 10]. This project asks the opposite question. It studies what a small network can do when it is trained from 252 hand-labelled chips on a laptop with a 4 GB GPU, and which design choices are worth their cost.

Research question. How well can a lightweight U-Net segment flood water in single-date Sentinel-1 imagery under these constraints, and which design choices matter? Four sub-questions structure the work:

- **RQ1, input channels.** Does adding the VV band and the VV/VH ratio to the VH band help?
- **RQ2, loss.** Water is a small minority of the pixels. Which loss copes best with that: binary cross-entropy (BCE), Dice, or their sum?
- **RQ3, encoder.** Is a light MobileNetV2 encoder, with or without ImageNet weights, better than a small U-Net trained from nothing, and what does each cost in parameters, time and memory?
- **RQ4, generalisation.** Does the performance hold on a flood (Bolivia) that the model has never seen?

Contributions.

1. A complete and reproducible pipeline for Sentinel-1 flood segmentation that runs on a 4 GB laptop GPU: 3 training runs of about 6 minutes each, with every number in this report generated from a saved result file.

2. A controlled ablation of input channels, loss function and encoder, each with three random seeds and a frozen protocol in which the test data were scored once per run, together with the cost of each encoder.
3. An error analysis that shows where and why the models fail, an honest comparison with two threshold baselines and with published results, and three practical findings that are easy to miss: (to be written).

2 Related work

Thresholding. The classical way to map water in a radar image is to threshold the backscatter, with the threshold chosen automatically, for instance by Otsu's method, which picks the cut that best separates a histogram into two groups [4]. Rule-based chains of this kind are still used operationally, and they remain a serious reference: Bereczky et al. benchmarked convolutional networks against such a chain for Sentinel-1 water and flood mapping [5].

The dataset and its baseline. Sen1Floods11 [7] contains 4,831 Sentinel-1 chips of 512×512 pixels from eleven flood events. For 446 of them a water mask was drawn by hand; the others carry automatically generated ("weak") labels. Its authors trained a fully convolutional network with a ResNet-50 backbone on the different label sets and compared it with thresholding. Two of their conclusions matter here: a network can beat a threshold on radar data, and many weak labels gave a better model than few hand labels. This project uses only the hand-labelled chips, so it works in the small-data regime in which the authors' own network did less well.

Deep networks on Sen1Floods11. U-Net [6], an encoder–decoder with skip connections, is the standard segmentation architecture and most later work builds on it. Konapala et al. compared combinations of Sentinel-1 and Sentinel-2 inputs for U-Nets [11]; their radar-only U-Net is listed with a test IoU of 0.45 in the comparison table of Yadav et al. [8]. Katiyar et al. trained SegNet and U-Net models for near-real-time mapping [12]. Bai et al. fused Sentinel-1 and Sentinel-2 in a boundary-aware network [13]. Closest to this project, Yadav et al. proposed an "Attentive U-Net" with a ResNet-50 encoder initialised from ImageNet that takes exactly the three inputs used as the default here (VV, VH and their ratio) and adds spatial and channel attention; it reaches a test IoU of 0.672, and a second network that also receives elevation and a permanent-water mask reaches 0.695 [8]. The same authors later used a pre-flood image as a second input [14].

More data instead of bigger labels. Patel et al. showed that self- and semi-supervised pre-training on unlabelled imagery improves flood segmentation, especially under geographic shift [15]. Foundation models push this further: Prithvi, pre-trained on a large archive of optical imagery [9], reaches a water IoU of 0.80 on the Sen1Floods11 test chips after fine-tuning [16], and Schmalstieg et al. distilled a 300-million-parameter successor into a 0.7-million-parameter student for edge devices [10]. The fine-tuned Prithvi model reads the optical Sentinel-2 bands of the same chips, not the radar bands, so such results show what is possible when the sky is clear rather than what radar alone can do.

Small models and classical learners. MobileNetV2 [17] is a compact network built from depthwise-separable convolutions and inverted residual blocks, designed for phones; with ImageNet weights [18] it is a common light encoder. At the other end, Iselborn et al. trained classical learners such as gradient-boosted trees on per-pixel features; with radar features alone their models reach a pooled IoU of about 0.5 on the test chips [19], which is the level of a well-chosen threshold (Section 5). Their paper is also careful about a point that makes published numbers hard to compare: some papers pool all pixels of the test set before computing IoU ("total IoU"), others average the IoU of each chip ("mean IoU"), and the two can differ by almost a factor of two on this dataset.

How reliable are small differences? Phunjanna et al. re-evaluated surface-water models on Sen1Floods11 and found that close rankings change with the random seed and with how regions are weighted [20]. That is the reason every configuration in this project is trained with three seeds, and why a difference is only called real when it exceeds the seed-to-seed spread.

Where this project sits. The studies above mostly add something: attention, extra inputs, more data, larger encoders. Few isolate the effect of the three most basic choices (input channels, loss, encoder) for a small model under a fixed protocol with repeated seeds, and few report what the choices cost on consumer hardware. That is the gap this project fills.

3 Data

3.1 The hand-labelled subset of Sen1Floods11

All experiments use the 446 hand-labelled chips of Sen1Floods11, version 1.1 [7]. A chip is a 512×512-pixel tile of a Sentinel-1 scene at 10 m per pixel, about 5 km on a side, stored as a two-band image: the backscatter in decibels (dB) for the VV and the VH polarisation. Each chip comes with a mask of the same size in which every pixel is 1 (water), 0 (not water) or −1 (no data). The masks mark all surface water, permanent and flood water alike. For every event the dataset also holds a Sentinel-2 optical image taken within two days of the radar image, and the masks were drawn with the help of both [7, 8]. This matters later: water that is visible optically is not always dark in the radar image.

The chips come from eleven flood events between 2016 and 2019 (Table 1). The dataset's official split is used unchanged: 252 training, 89 validation and 90 test chips drawn from ten of the events, and the 15 chips of the eleventh, Bolivia, held out completely. The test chips measure performance on new places within familiar floods; the Bolivia chips measure performance on a flood the model has never seen (RQ4). Before any modelling it was checked that all 446 image and label files exist, that the four split lists cover every chip exactly once, and that no Bolivia chip appears in the other three.

Table 1. The eleven flood events of the hand-labelled subset and the official split. Dates and orbit direction are taken from the dataset's metadata file.

Event	Radar image	Orbit	Hand-labelled chips	Train	Validation	Test	Held out
Ghana	2018-09-18	ascending	53	31	11	11	–
India	2016-08-12	descending	68	40	14	14	–
Mekong	2018-08-05	ascending	30	18	6	6	–
Nigeria	2018-09-21	ascending	18	10	4	4	–
Pakistan	2017-06-28	descending	28	16	6	6	–
Paraguay	2018-10-31	descending	67	39	14	14	–
Somalia	2018-05-07	ascending	26	15	5	6	–
Spain	2019-09-17	descending	30	18	6	6	–
Sri-Lanka	2017-05-30	descending	42	24	9	9	–
USA	2019-05-22	ascending	69	41	14	14	–
Bolivia	2018-02-15	descending	15	–	–	–	15
Total			**446**	**252**	**89**	**90**	**15**

3.2 What the training data look like

All statistics that influenced a design decision were computed on the training chips only. Figure 1 shows six of them. Three properties of the data shape the rest of the project.

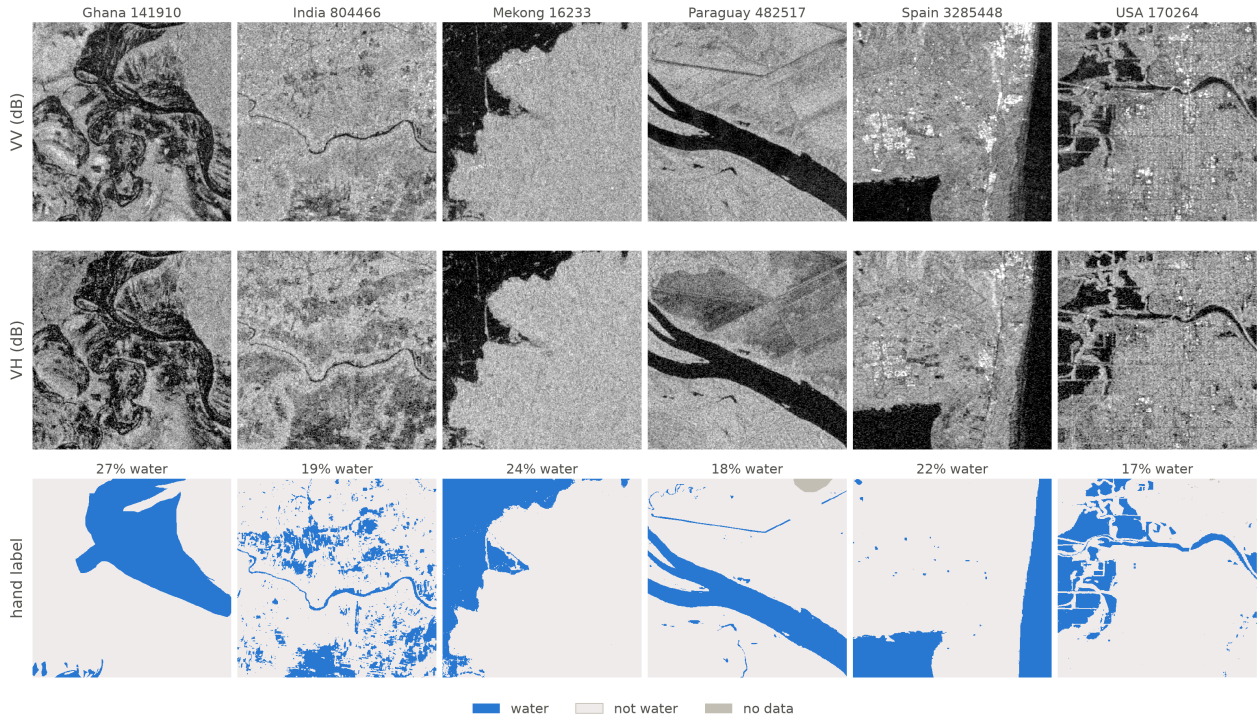


Figure 1. Six training chips from six events: VV, VH and the hand label. Both radar bands use the same fixed dB range in every panel. Open water is dark, more cleanly so in VH than in VV. The masks contain thin rivers and ragged shorelines only a few pixels wide.

Water is rare. Water covers 9.5% of the labelled training pixels. 32 of the 252 training chips contain no water at all and 159 contain less than 5%. A model that never predicts water would be about 90% accurate, so accuracy is not reported; and a loss that treats all pixels alike is dominated by the easy "not water" pixels, which is why the loss is one of the factors studied (RQ2).

One number does not separate water from land. Figure 2 shows the distribution of both bands for the two classes. Water is darker, but the distributions overlap: the shared area of the two normalised histograms is 0.34 in VH and 0.44 in VV. In VV the water distribution has a second hump that lies largely under the land distribution; the likely cause is water roughened by wind or mixed with vegetation, to which VV is sensitive. A threshold on VH is therefore a fair but limited baseline, and there is room for a model that looks at both bands and at the neighbourhood of a pixel (RQ1).

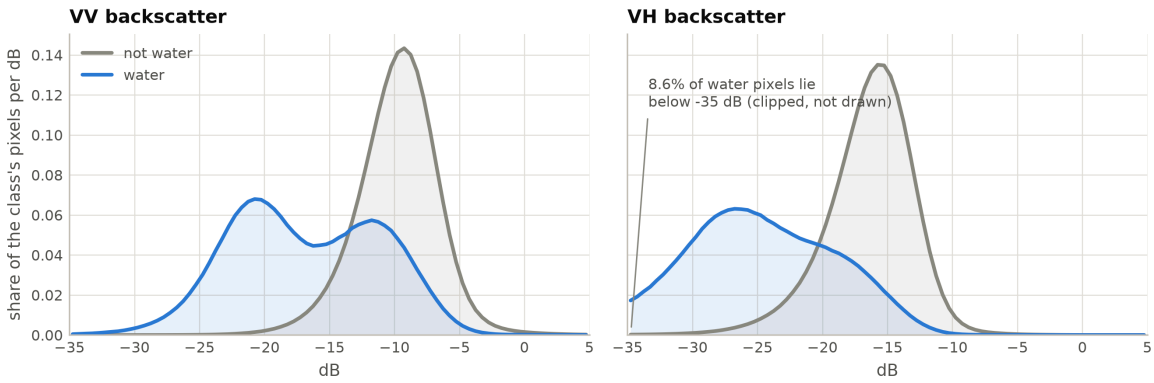


Figure 2. Backscatter of the training pixels, water against not water, each class normalised to unit area. The classes overlap in both bands, less in VH than in VV.

Part of the data is missing. 13.3% of the training pixels have no label, and 2.0% of the radar pixels have no value (NaN), nearly always where the label is missing too. Unlabelled pixels are excluded from the loss and from every metric. A small number of pixels have a label but no radar value; the largest such case is one test chip whose radar image is entirely empty although a fifth of it is labelled as water. No method can classify such pixels from the image. They are kept in the reported scores, following the usual convention that every labelled pixel counts, and Section 5 also gives the scores without them.

Table 2 summarises the four splits. The water share differs between them, and between events it differs far more, so a score pooled over all pixels and a score averaged over chips will not agree; both are reported.

Table 2. The four splits. Only the training column was used for design decisions; the other columns are label bookkeeping.

Split	Chips	Events	Labelled pixels (millions)	Water share of labelled pixels	Pixels without label	Chips without water	Labelled pixels without radar value
Train	252	10	57.3	9.5%	13.3%	32	24,150
Validation	89	10	20.3	11.0%	13.0%	9	20,571
Test	90	10	20.5	12.5%	13.0%	6	73,408
Bolivia (held out)	15	1	2.9	15.9%	27.1%	0	43,381

3.3 Preprocessing

Every chip goes through the same function, in training, in evaluation and in the inference demo:

1. Both bands are clipped to [−35, 5] dB. This touches 0.45% of the training values, mostly very dark water in VH.
2. A third channel is added, the VV/VH ratio. On the logarithmic dB scale a ratio is a difference, so the channel is VV − VH, computed from the clipped values.
3. Each channel is standardised with the mean and standard deviation of the training chips (VV −10.39 ± 4.03 dB, VH −17.20 ± 4.55 dB, ratio 6.80 ± 3.31 dB). The validation and test chips play no part in these statistics.
4. Missing radar values are set to 0, which after standardising is the average training pixel.

Depending on the configuration the model receives VH alone, VV and VH, or all three channels.

4 Method

4.1 Threshold baselines (experiment E0)

The simplest flood mapper is one rule: a pixel is water if its VH backscatter is below a threshold. Two thresholds were used.

Otsu (run R00, the planned baseline). Otsu's method [4] was applied once to the histogram of all VH values of the training chips, without labels, and gave −19.95 dB. A single global threshold was preferred to one threshold per chip because Otsu always splits a histogram in two: on the 32 training chips that contain no water, a per-chip threshold marks a median of 43% of the chip as water.

Tuned threshold (run R00b, an added reference). A check on the training and validation chips showed that Otsu's cut is not the best one. The threshold with the highest IoU on the training labels is −23.60 dB. It was added as a second, stronger reference so that the networks are not compared only with a baseline that is easy to beat. It was fitted on training labels alone.

4.2 U-Net with three encoders

A U-Net [6] has an encoder that reduces the image step by step to a small, information-rich representation, a decoder that brings it back to full size, and skip connections that hand the encoder's fine detail to the decoder at each scale. All models here end in one sigmoid output per pixel, the probability of water, and accept any image size divisible by 32, so the same model is trained on 256×256 crops and applied to full 512×512 chips.

- **Scratch U-Net.** Five levels with 16, 32, 64, 128 and 256 filters. Each level is two 3×3 convolutions, each followed by batch normalisation [21] and ReLU. Max-pooling halves the resolution on the way down; on the way up the feature map is upsampled by two, concatenated with the skip connection and passed through the same pair of convolutions. 1.97 million parameters.
- **MobileNetV2 U-Net (the default).** The encoder is MobileNetV2 [17] up to its last block at 1/32 of the input size. Four intermediate feature maps, at 1/2, 1/4, 1/8 and 1/16 scale, serve as skip connections. The decoder has four blocks with 256, 128, 64 and 32 filters, built like the scratch decoder, and a final upsampling stage with 16 filters; MobileNetV2 halves the resolution in its very first layer, so there is no skip connection at full resolution. 5.42 million parameters, of which 3.58 million are in the decoder. The encoder weights are initialised randomly.
- **MobileNetV2 U-Net with ImageNet weights.** The same network with the encoder initialised from ImageNet classification weights [18]. This needs exactly three input channels, which the default input provides. The weights were learned on colour photographs; the network is given standardised radar channels instead and all layers are trained, so whether the weights still help is part of RQ3.

4.3 Losses

Let p_i be the predicted water probability of pixel i, $y_i \in \{0, 1\}$ its label, and V the set of labelled pixels in a batch. Unlabelled pixels are left out of every sum.

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{|V|} \sum_{i \in V} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_{i \in V} p_i y_i + \varepsilon}{\sum_{i \in V} p_i + \sum_{i \in V} y_i + \varepsilon}, \qquad \varepsilon = 10^{-6}$$

Binary cross-entropy judges every pixel separately, so with rare water it is dominated by land pixels. The Dice loss [22] measures the overlap between predicted and true water and ignores how many land pixels were easy. The default is their sum, $\mathcal{L}_{\mathrm{BCE}} + \mathcal{L}_{\mathrm{Dice}}$.

4.4 Training

One epoch consists of 4 random 256×256 crops of every training chip, each randomly flipped and rotated by a multiple of 90°, with image and mask transformed identically: 1,008 crops, or 126 batches of 8. Models were trained with Adam [23] at a learning rate of 10^{-3} for at most 60 epochs. After every epoch the model was scored on the full validation chips. The validation water IoU drove three mechanisms: the learning rate was halved after 5 epochs without improvement (down to 10^{-5}), training stopped after 15 epochs without improvement, and the weights of the best epoch were kept. Training used mixed precision (16-bit arithmetic inside the network, 32-bit for the output and the loss).

These defaults were fixed after a five-epoch smoke test and a first full run that was inspected on validation data only (Section 4.7), then tagged in the code repository before any other model was trained.

4.5 Metrics

With TP, FP and FN the numbers of true-positive, false-positive and false-negative water pixels among the labelled pixels, and a decision threshold of 0.5 on the predicted probability:

$$\mathrm{IoU} = \frac{TP}{TP + FP + FN}, \qquad \mathrm{precision} = \frac{TP}{TP + FP}, \qquad \mathrm{recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2\,TP}{2\,TP + FP + FN}$$

The primary metric is the **pooled water IoU**: TP, FP and FN are summed over all chips of a split before dividing once. Precision, recall and F1 are pooled in the same way. As a secondary metric the **mean per-chip IoU** is the plain average of the chips' individual IoU values; chips with no water in the label and none predicted have an undefined IoU and are left out of that average. The two metrics answer different questions. The pooled IoU is dominated by chips with much water; the per-chip mean gives a chip with a small pond the same weight as a chip that is half flooded.

The same tested metric code scores the threshold baselines and every network.

4.6 Experimental design and protocol

Table 3 lists the runs. Each study changes one factor and keeps everything else at the default, which serves as the shared reference of all three studies. Every configuration was trained three times with seeds 0, 1 and 2; the seed fixes the initial weights and the sequence of crops.

Table 3. The experiments. Each study varies one factor; the default configuration (bold) is shared.

Study	Question	Configurations	Runs
E0	Is learning better than a threshold?	Otsu on VH; tuned VH threshold (added)	R00, R00b
E1	RQ1, input channels	VH · VV+VH · **VV+VH+ratio**	R04–R09, R01–R03
E2	RQ2, loss	BCE · Dice · **BCE+Dice**	R10–R15, R01–R03
E3	RQ3, encoder	Scratch U-Net · **MobileNetV2** · MobileNetV2 + ImageNet	R16–R21, R01–R03

Three rules were followed throughout.

1. **Choices are made on validation data only.** Stopping, checkpoint selection and the choice of the "best" model use the 89 validation chips.
2. **Test and Bolivia are scored once per finished run.** No setting was changed after a test score had been seen. The first model was inspected on validation data, the defaults were frozen, and only then was it scored on test.
3. **Only the factor under study changes.** Splits, seeds and all other settings are identical across configurations; a check at the start of each experiment confirms that the training and scoring code is byte-identical to the frozen version.

Reading differences. Results are reported as mean ± standard deviation over the three seeds. A difference between two configurations is treated as real only if it is larger than the larger of their two standard deviations. For the main model, a second kind of uncertainty is also given: a 95% bootstrap interval obtained by resampling the test chips, which shows how much the score depends on which chips happen to be in the test set.

4.7 Implementation

The code is written in Python with TensorFlow 2.21.0 and Keras 3.15.1 and ran on a laptop with an NVIDIA GeForce RTX 3050 Laptop GPU (4 GB) under WSL2 with 7.5 GB of RAM. Every run trains in its own process and writes its full configuration, its training history, the per-chip scores and the code version next to the model, so each number in this report can be traced to a file. Two findings from the smoke test shaped the set-up and are worth passing on.

Compilation. Compiling the training step with XLA, TensorFlow's just-in-time compiler, reduced the time per step from 414 ms to 62 ms on this machine. Without it the planned runs would not have fitted the time available.

Batch-normalisation momentum. The MobileNetV2 that ships with Keras uses a batch-normalisation momentum of 0.999, which means the running statistics used at validation and test time need thousands of updates to catch up with the weights. An epoch here has 126 updates. In the smoke test the training loss fell normally while the validation IoU stayed at exactly 0.00 for all five epochs; with a momentum of 0.99 in every layer it reached 0.60 (Figure 3). Because checkpointing, learning-rate reduction and early stopping all depend on the validation score, the momentum was set to 0.99 for every model. Anyone who trains a Keras application model from scratch with few updates per epoch is likely to meet the same problem.

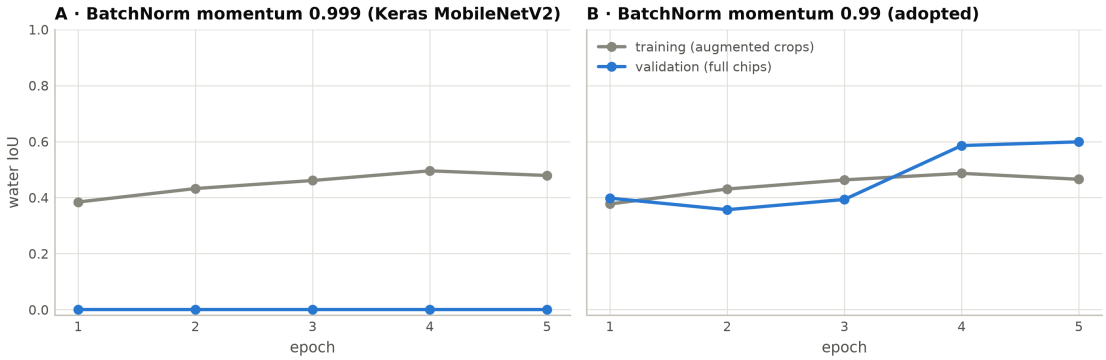


Figure 3. The same five-epoch training run with two batch-normalisation momenta. Training behaves alike in both; the validation score only follows when the running statistics can keep up.

5 Results

(to be written)

References

1. Centre for Research on the Epidemiology of Disasters (CRED) and UN Office for Disaster Risk Reduction (UNDRR). The human cost of disasters: an overview of the last 20 years (2000–2019). *UNDRR*, 2020. https://www.undrr.org/publication/human-cost-disasters-overview-last-20-years-2000-2019
2. B. Tellman, J. A. Sullivan, C. Kuhn, A. J. Kettner, C. S. Doyle, G. R. Brakenridge, T. A. Erickson, and D. A. Slayback. Satellite imaging reveals increased proportion of population exposed to floods. *Nature*, 596(7870):80–86, 2021. doi:10.1038/s41586-021-03695-w
3. R. Torres, P. Snoeij, D. Geudtner, D. Bibby, M. Davidson, E. Attema, P. Potin, B. Rommen, N. Floury, M. Brown et al. GMES Sentinel-1 mission. *Remote Sensing of Environment*, 120:9–24, 2012. doi:10.1016/j.rse.2011.05.028
4. N. Otsu. A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979. doi:10.1109/TSMC.1979.4310076
5. M. Bereczky, M. Wieland, C. Krullikowski, S. Martinis, and S. Plank. Sentinel-1-Based Water and Flood Mapping: Benchmarking Convolutional Neural Networks Against an Operational Rule-Based Processing Chain. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 15:2023–2036, 2022. doi:10.1109/JSTARS.2022.3152127
6. O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 9351:234–241, 2015. doi:10.1007/978-3-319-24574-4_28
7. D. Bonafilia, B. Tellman, T. Anderson, and E. Issenberg. Sen1Floods11: A Georeferenced Dataset to Train and Test Deep Learning Flood Algorithms for Sentinel-1. *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 835–845, 2020. doi:10.1109/CVPRW50498.2020.00113
8. R. Yadav, A. Nascetti, and Y. Ban. Deep attentive fusion network for flood detection on uni-temporal Sentinel-1 data. *Frontiers in Remote Sensing*, 3:1060144, 2022. doi:10.3389/frsen.2022.1060144
9. J. Jakubik, S. Roy, C. E. Phillips, P. Fraccaro, D. Godwin, B. Zadrozny, D. Szwarcman, C. Gomes et al. Foundation Models for Generalist Geospatial Artificial Intelligence. arXiv preprint arXiv:2310.18660, 2023.
10. F. Schmalstieg, K. Mueller, and W. Samek. Cross-Architecture Foundation-Model Distillation for Edge Flood Segmentation. arXiv preprint arXiv:2609.20441, 2026.
11. G. Konapala, S. V. Kumar, and S. K. Ahmad. Exploring Sentinel-1 and Sentinel-2 diversity for flood inundation mapping using deep learning. *ISPRS Journal of Photogrammetry and Remote Sensing*, 180:163–173, 2021. doi:10.1016/j.isprsjprs.2021.08.016
12. V. Katiyar, N. Tamkuan, and M. Nagai. Near-Real-Time Flood Mapping Using Off-the-Shelf Models with SAR Imagery and Deep Learning. *Remote Sensing*, 13(12):2334, 2021. doi:10.3390/rs13122334
13. Y. Bai, W. Wu, Z. Yang, J. Yu, B. Zhao, X. Liu, H. Yang, E. Mas, and S. Koshimura. Enhancement of Detecting Permanent Water and Temporary Water in Flood Disasters by Fusing Sentinel-1 and Sentinel-2 Imagery Using Deep Learning Algorithms: Demonstration of Sen1Floods11 Benchmark Datasets. *Remote Sensing*, 13(11):2220, 2021. doi:10.3390/rs13112220
14. R. Yadav, A. Nascetti, and Y. Ban. Attentive Dual Stream Siamese U-net for Flood Detection on Multi-temporal Sentinel-1 Data. *IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, 2022. doi:10.1109/IGARSS46834.2022.9883132
15. C. Patel, S. Sharma, V. J. Pasquarella, and V. Gulshan. Evaluating Self and Semi-Supervised Methods for Remote Sensing Segmentation Tasks. arXiv preprint arXiv:2111.10079, 2021.
16. J. Jakubik, P. Fraccaro, D. Oliveira Borges, M. Muszynski, K. Weldemariam, B. Zadrozny, R. Ganti, and K. Mukkavilli. Prithvi 100M flood mapping: model card of Prithvi-EO-1.0-100M-sen1floods11. Hugging Face, https://huggingface.co/ibm-nasa-geospatial/Prithvi-100M-sen1floods11, 2023. doi:10.57967/hf/0973

17. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018. doi:10.1109/CVPR.2018.00474

18. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255, 2009. doi:10.1109/CVPR.2009.5206848

19. K. Iselborn, M. Stricker, T. Miyamoto, M. Nuske, and A. Dengel. On the Importance of Feature Representation for Flood Mapping using Classical Machine Learning Approaches. arXiv preprint arXiv:2303.00691, 2023.

20. K. Phunjanna, K. Karacs, and C. Ngamkhanong. A Controlled Evaluation of Model Rankings and Input Reliance in Surface Water Segmentation. arXiv preprint arXiv:2608.30895, 2026.

21. S. Ioffe and C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

22. F. Milletari, N. Navab, and S.-A. Ahmadi. V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation. *International Conference on 3D Vision (3DV)*, pp. 565–571, 2016. doi:10.1109/3DV.2016.79

23. D. P. Kingma and J. Ba. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*, 2015.